

Altasker MVP — 18 Main Flows (Scope-Reduced · 28 Tables)

Cross-Table CRUD Grounding · State Machines · Endpoint Mapping

Purpose: Definitive flow reference for teacher assessment. Every step is grounded to a specific table, column, state transition, and API endpoint from the 28-table MVP schema.

Last updated: June 2026

Conventions: **[LEDGER]** = wallet_transactions row written. **[API]** = external service call. Tables in **bold** on first reference. \$0.x = Master Reference Sheet section.

Group 1 — Onboarding & Account Setup

MF-1: Client (CEO) Registration & Subscription

Overview

Registers a CLIENT/CEO account, creates their internal wallet and permanent top-up VA, then activates Client Pro subscription via direct wallet deduction.
References: \$0.7 (RBAC), \$0.8 (Payment Architecture), \$0.9 (Feature Gates).

Tables touched (5): users, client_profiles, wallets, virtual_accounts, wallet_transactions

Endpoints: POST /auth/register, POST /wallet/topup-qr, POST /subscriptions/activate, POST /webhooks/sepay/ipn

ASCII Swimlane





		INSERT wallet_transactions:	
		type:"TOP_UP", amt:500000	
		ref: transfer_reference	
		COMMIT	
		e. Return 200 OK to SePay	
	+<-----	f. Push notification	
[16] Dashboard: balance = 500,000			
= PHASE C: SUBSCRIPTION ACT. =			
[17] Clicks "Activate Client Pro"			
(500K VND / 6 mo)			
	+----->		
		[18] POST /subscriptions/activate	
		{role_type:"client"}	
		Guard 1: users.sub_client_tier	
		IF "pro" AND not expired	
		→ 409 ALREADY_SUBSCRIBED	
		Guard 2: wallets.available_	
		balance >= 500000?	
		NO → 422 INSUFFICIENT_BALANCE	
		[balance ≥ 500K]	
		[19] DB TX (atomic):	
		UPDATE wallets SET	
		available_balance -= 500000	
		INSERT wallet_transactions:	
		type:"SUBSCRIPTION"	
		amt:500000	
		ref:"SUB-{"user_id":client"	
		UPDATE users SET	
		sub_client_tier='pro'	
		sub_client_expires_at=	
		now()+6 months	
		COMMIT	
		[20] Reissue JWT (updated claims)	
		Notify: "Pro active until {date}"	
	+<-----		
[21] ✓ CLIENT PRO ACTIVE			
Unlocked: F2 Elicitation,			
F4 Matching, Artifact B,			
Full Seam Gap Maps			
Balance: 0 VND			
+-----+-----+-----+			

Step Detail Table

Step	Actor	Action	Tables (CRUD)	State Change	Endpoint
1-2	CEO	Fill registration form	—	—	GET /register
3	NestJS	Validate input (email unique, pwd≥8, phone)	users (R)	—	POST /auth/register
4	NestJS	Atomic insert: user + client_profile + wallet	users (C), client_profiles (C), wallets (C)	New: user.roles= ["CLIENT_CEO"], wallet available=0	—
5-6	NestJS→SePay	Create WALLET_TOPUP VA	— (SePay-side)	—	POST SePay VA API
7	NestJS	Store VA record	virtual_accounts (C)	VA status=ACTIVE	—
8	NestJS	Generate JWT	—	—	—
9	CEO	Dashboard loads free tier	users (R), wallets (R)	—	GET /dashboard
10-11	CEO→NestJS	Request top-up QR	virtual_accounts (R)	—	POST /wallet/topup-qr
12-14	CEO→Bank→SePay	External bank transfer + IPN	—	—	External
15	NestJS	IPN: verify HMAC → resolve VA → idempotency → credit wallet	virtual_accounts (R), wallets (U), wallet_transactions (C)	wallet.available: 0→500K	POST /webhooks/sepay/ipn
16	CEO	Dashboard reflects new balance	wallets (R)	—	GET /wallet
17-18	CEO→NestJS	Activate subscription; guards pass	users (R), wallets (R)	—	POST /subscriptions/activate
19	NestJS	Atomic: debit wallet + ledger + update user tier	wallets (U), wallet_transactions (C), users (U)	wallet.available: 500K→0, users.sub_client_tier: free→pro	—
20-21	NestJS→CEO	Reissue JWT, notify	users (R)	—	—

Cross-Table CRUD Map

```
users —(1:1)→ client_profiles      [C in step 4]
users —(1:1)→ wallets              [C in step 4, R/U in steps 15,19]
users      .subscription_client_tier [R step 18, U step 19]
users      .sub_client_expires_at   [U step 19]
wallets —(1:N)→ wallet_transactions [C in steps 15,19]
virtual_accounts .entity_id → users.id (polymorphic) [C step 7, R steps 11,15]
wallet_transactions .reference_id UNIQUE index [idempotency in step 15]
```

State Machine Reference

- **Subscription state (\$0.9):** users.subscription_client_tier: free → pro (step 19); stored directly on users — no user_subscriptions table in MVP
- **VA state (\$0.8):** virtual_accounts.status: ACTIVE (permanent for WALLET_TOPUP)
- **Wallet balance invariants:** available_balance >= 0 (CHECK constraint), locked_balance >= 0 (CHECK constraint)

MF-2: Expert Registration, Profile & Tier 1→2 Verification

Overview

Registers an EXPERT account, creates taxonomy profile with domain depths and seam claims (Tier 1 Claimed), then optionally submits portfolio evidence for LLM auto-verification to Tier 2 (Evidence-backed). Links bank account via Bank Hub Hosted Link for future chi hợ withdrawals. References: \$0.1 (Domains), \$0.2 (Seams), \$0.4 (2-Tier Verification).

Tables touched (8): users, expert_profiles, wallets, virtual_accounts, wallet_transactions, expert_domain_depths, expert_seam_claims, portfolio_submissions, platform_decisions

Endpoints: POST /auth/register, PUT /expert-profile, POST /expert-profile/domains, POST /expert-profile/seams, POST /portfolio-submissions, GET /portfolio-submissions/{id}/eval-result, POST /bank-hub/initiate-link, POST /webhooks/sepay/bank-linked



```

to Tier 2"
Guard: Expert Pro required
+----->|
[12] Guard: users.sub_expert_tier
IF "free" → 403 SUBSCRIPTION_
REQUIRED
IF "pro" → proceed

[13] Fills portfolio form for
seam A*C:
• project_description
• decision_points (structured)
+----->|

[14] POST /portfolio-submissions
Throttle check:
SELECT submission_count,
locked_until FROM
expert_seam_claims
WHERE expert_id=? AND
seam_code='A*C'
IF submission_count≥5 AND
locked_until>now()
→ 429 TOO_MANY_ATTEMPTS
INSERT portfolio_submissions:
expert_id, seam_claim_id,
project_description,
decision_points,
status:'PENDING',
submitted_at:now()
UPDATE expert_seam_claims SET
submission_count += 1

[15] FastAPI LLM extraction call
→ POST /llm/portfolio-eval
{project_description,
decision_points, seam_code}

FastAPI runs LLM rubric:
• All required signal types?
• Confidence score computed
• Returns {confidence, passed,
gap_advisory?}

[16] DB TX:
UPDATE portfolio_submissions SET
llm_confidence = {score},
evaluated_at = now(),
status = 'APPROVED' | 'REJECTED'

IF confidence ≥ 0.85 AND passed:
UPDATE expert_seam_claims SET
verification_tier =
'EVIDENCE_BACKED'
(confidence factor: 0.55 per
$0.4)
INSERT platform_decisions:
decision_type:'SEAM_TIER_UPGRADE'
OR 'PORTFOLIO_EVAL',
entity_type:'expert_seam_claims',
entity_id:claim_id,
llm_confidence:{score},
decision:'APPROVED' | 'REJECTED',
advisory_note:{gap_advisory}

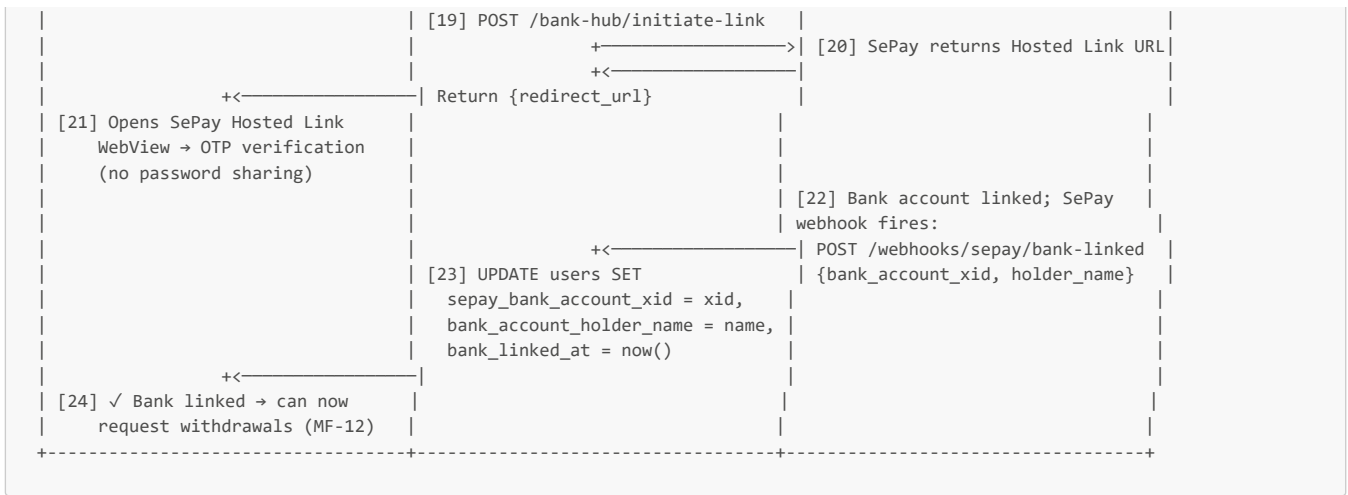
IF rejected AND submission_count
≥ 5:
UPDATE expert_seam_claims SET
locked_until = now()+30days

+<-----|
[17] Result:
✓ A*C → Tier 2 EVIDENCE_BACKED
X A*D → remains Tier 1 CLAIMED
(gap advisory shown for
resubmission)

== PHASE D: BANK ACCOUNT LINK ==

[18] Clicks "Link Bank Account"
+----->|

```



Step Detail Table

Step	Actor	Action	Tables (CRUD)	State Change	Endpoint
1-3	Expert→NestJS	Register account	users (C), expert_profiles (C), wallets (C), virtual_accounts (C)	New expert user	POST /auth/register
5-6	Expert→NestJS	Set domain depths	expert_domain_depths (C/U via UPSERT)	verification_tier='CLAIMED' (Tier 1)	POST /expert-profile/domains
7-8	Expert→NestJS	Claim seams	expert_seam_claims (C)	verification_tier='CLAIMED'	POST /expert-profile/seams
9-10	Expert→NestJS	Set stack tags + engagement model + archetype history	expert_profiles (U)	—	PUT /expert-profile
11-12	Expert→NestJS	Tier 2 upgrade attempt; subscription guard	users (R)	—	POST /portfolio-submissions
13-14	Expert→NestJS	Submit portfolio evidence	portfolio_submissions (C), expert_seam_claims (U: submission_count++)	status='PENDING'	—
15	NestJS→FastAPI	LLM portfolio evaluation	—	—	POST /llm/portfolio-eval (internal)
16	FastAPI→NestJS	Process LLM result	portfolio_submissions (U), expert_seam_claims (U), platform_decisions (C)	seam verification_tier: CLAIMED→EVIDENCE_BACKED (if ≥0.85)	—
18-19	Expert→NestJS	Initiate Bank Hub link	—	—	POST /bank-hub/initiate-link
20-21	Expert→SePay	Complete OTP in Hosted Link	—	—	SePay WebView
22-23	SePay→NestJS	Bank linked webhook	users (U)	sepay_bank_account_xid set	POST /webhooks/sepay/bank-linked

Cross-Table CRUD Map

```

users —(1:1)—> expert_profiles [C step 3, U step 23]
expert_profiles —(1:N)—> expert_domain_depths [C step 6]
expert_profiles —(1:N)—> expert_seam_claims [C step 8, U steps 14,16]
expert_seam_claims —(1:N)—> portfolio_submissions [C step 14, U step 16]
portfolio_submissions → platform_decisions [C step 16 – polymorphic entity_id]
expert_profiles.stack_tags_json (JSONB) [U step 10 – absorbs cut expert_stack_tags table]
expert_profiles.archetype_history_json (JSONB) [U step 10 – self-declared for cold start]

```

Key scope-reduction note: No `scenario_assessments`, `scenario_responses`, or `expert_seam_outcome_signals` tables. Expert verification maxes at Tier 2 (EVIDENCE_BACKED, confidence 0.55). Tiers 3-4 deferred to Phase 2.

MF-3: Tech Team Account Creation via Handoff Link

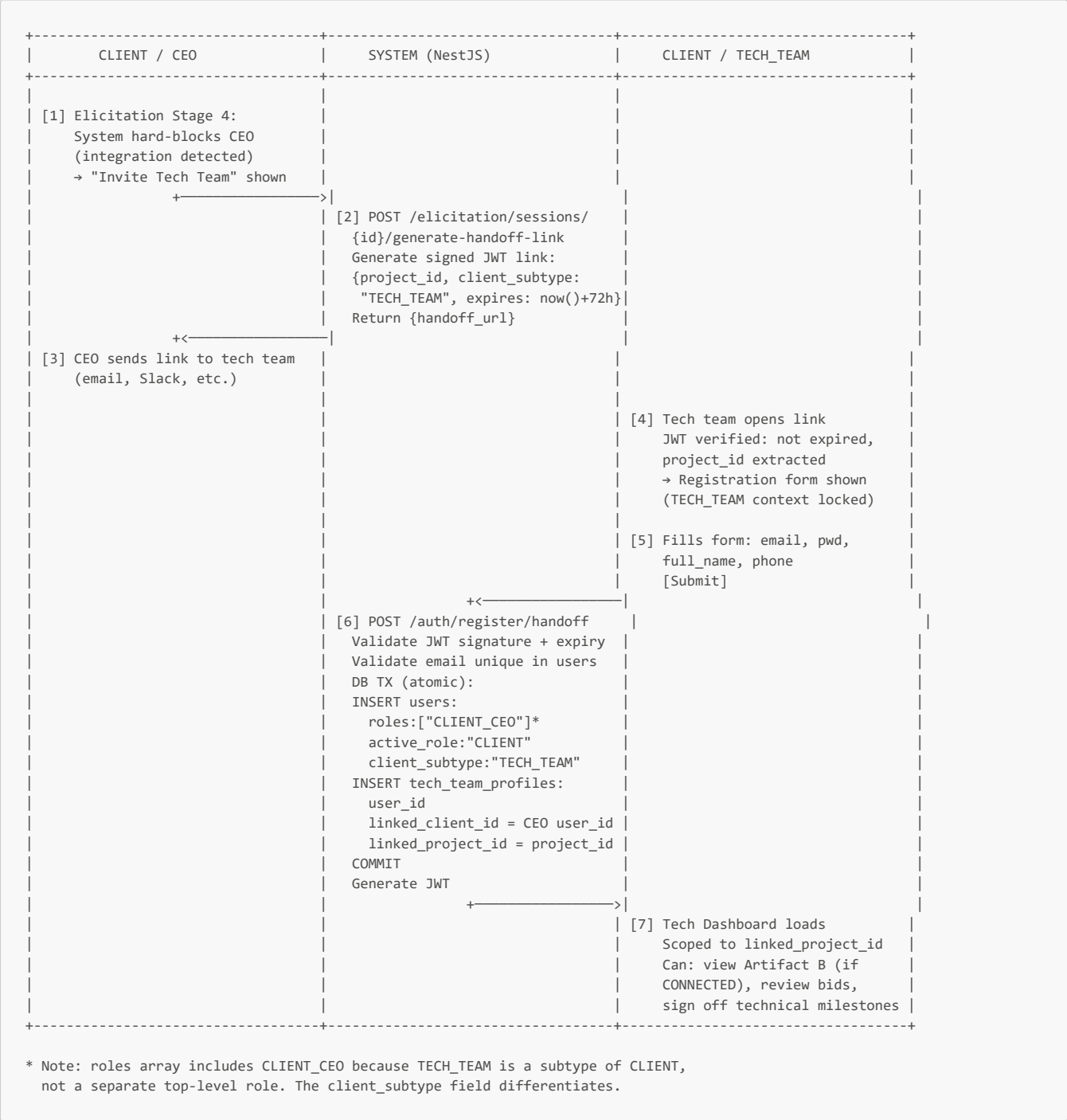
Overview

CEO generates a signed handoff link during Elicitation Stage 4. The link is sent to the client's technical team member, who uses it to register as CLIENT/TECH_TEAM. This is the **only** way a TECH_TEAM account is created — no self-registration. References: \$0.7 (RBAC — TECH_TEAM creation constraint).

Tables touched (3): `users`, `tech_team_profiles`, `projects`

Endpoints: `POST /elicitation/sessions/{id}/generate-handoff-link`, `POST /auth/register/handoff`

ASCII Swimlane



Step Detail Table

Step	Actor	Action	Tables (CRUD)	State Change	Endpoint
------	-------	--------	---------------	--------------	----------

Step	Actor	Action	Tables (CRUD)	State Change	Endpoint
1-2	CEO→NestJS	Generate handoff link (signed JWT with project_id + 72h expiry)	projects (R — validate project exists)	—	POST /elicitation/sessions/{id}/generate-handoff-link
3	CEO	Sends link externally	—	—	—
4-5	TECH_TEAM	Opens link, fills form	—	—	GET /register?token={jwt}
6	NestJS	Validate handoff JWT + register TECH_TEAM	users (C), tech_team_profiles (C)	New TECH_TEAM user, scoped to one project	POST /auth/register/handoff
7	TECH_TEAM	Tech Dashboard loads	tech_team_profiles (R — get linked_project_id)	—	GET /tech-dashboard

Cross-Table CRUD Map

```
users —(1:1)—> tech_team_profiles      [C step 6]
tech_team_profiles.linked_client_id → users.id (CEO) [C step 6]
tech_team_profiles.linked_project_id → projects.id [C step 6 — scope lock]
tech_team_profiles linked_project_id INDEX [R step 7 — fast project scoping]
```

Key constraint: TECH_TEAM account is **locked to one project** via `linked_project_id`. No self-registration. This enforces the §0.7 RBAC rule that only CEOs can initiate the tech team relationship.

Group 2 — Path A: Project-Based Flow

MF-4: AI Elicitation Engine (5-Stage)

Overview

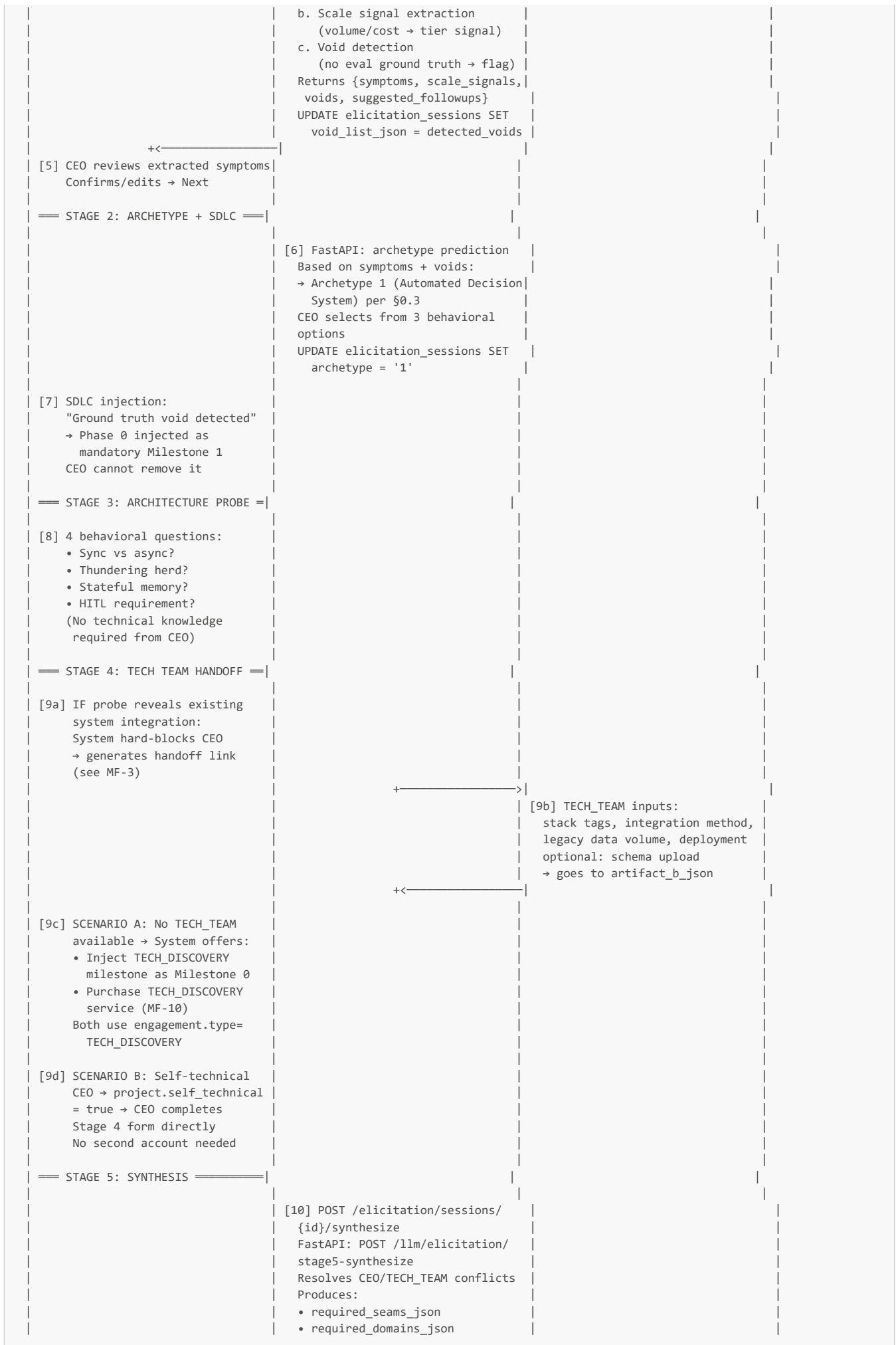
The platform's primary AI innovation. Transforms a CEO's raw symptom description into a taxonomy-grounded capability footprint (Artifact A + Artifact B) through a 5-stage conversational diagnostic. Supports Scenario A (no TECH_TEAM → TECH_DISCOVERY) and Scenario B (self-technical CEO). References: §0.1 (Domains), §0.2 (Seams), §0.3 (Archetypes + Tiers), §0.6 (Elicitation + Spec state machines).

Tables touched (4): `elicitation_sessions`, `projects`, `platform_decisions`, `users`

Endpoints: `POST /elicitation/sessions`, `PUT /elicitation/sessions/{id}/stage`, `POST /elicitation/sessions/{id}/synthesize`, `POST /llm/elicitation/*` (FastAPI internal)

ASCII Swimlane







Step Detail Table

Step	Actor	Action	Tables (CRUD)	State Change	Endpoint
1-2	CEO→NestJS	Start elicitation session; subscription guard	users (R — guard), elicitation_sessions (C)	session.state=IN_PROGRESS, current_stage=1	POST /elicitation/sessions
3-4	CEO→FastAPI	Symptom intake; LLM extraction	elicitation_sessions (U — void_list_json)	—	POST /llm/elicitation/stage1-extract
5-6	CEO→FastAPI	Archetype confirmation + SDLC injection	elicitation_sessions (U — archetype)	—	PUT /elicitation/sessions/{id}/stage
7	System	Void injection (Phase 0 mandatory)	elicitation_sessions (U — void_list_json)	—	—
8	CEO→NestJS	Architecture probe (4 behavioral questions)	elicitation_sessions (U — current_stage=3)	—	PUT /elicitation/sessions/{id}/stage
9a-b	CEO/TECH_TEAM	Tech team handoff (see MF-3)	users (C), tech_team_profiles (C)	—	POST /auth/register/handoff
9c	CEO→NestJS	Scenario A: TECH_DISCOVERY offer	elicitation_sessions (U — scenario_type='SCENARIO_A')	—	—

Step	Actor	Action	Tables (CRUD)	State Change	Endpoint
9d	CEO→NestJS	Scenario B: self_technical=true	<code>elicitation_sessions</code> (U) — <code>scenario_type='SCENARIO_B'</code>	—	<code>PUT /elicitation/sessions/{id}/stage</code>
10	NestJS→FastAPI	Stage 5 synthesis	—	—	<code>POST /llm/elicitation/stage5-synthesize</code>
11	NestJS	Auto-publish quality gate + project creation	<code>projects</code> (C), <code>elicitation_sessions</code> (U), <code>platform_decisions</code> (C)	<code>projects.state: PUBLISHED</code> or <code>RETURNED_TO_CLIENT</code> ; <code>session.state: COMPLETED</code> or <code>RETURNED</code>	<code>POST /elicitation/sessions/{id}/synthesize</code>
12	CEO	View result	<code>projects</code> (R)	—	—

State Machine References

Elicitation session (§0.6):

```
IN_PROGRESS (current_stage advances 1→2→3→4→5)
  → COMPLETED (Stage 5 synthesis produced project; project.state=PUBLISHED)
  → RETURNED (auto-publish gate failed; re-enter at current_stage)
  → ABANDONED (CEO navigated away; can resume)
```

Spec/Project (§0.6):

```
DRAFT → [auto-publish quality gate] → PUBLISHED
      ↓ (fail)
      RETURNED_TO_CLIENT (LLM advisory note; re-enter at specific void)
PUBLISHED → SUSPENDED (admin emergency only)
```

Cross-table dependency:

```
elicitation_sessions.user_id → users.id
projects.elicitation_session_id → elicitation_sessions.id
projects.client_id → users.id
projects self-contains: required_seams_json, required_domains_json,
  milestone_framework_json, artifact_a_json, artifact_b_json
(5 JSONB columns replace 3 cut tables: capability_footprints, artifact_a, artifact_b)
```

MF-5: AI Matching & Shortlisting (2-Tier Confidence)

Overview

Composite scoring engine ranks experts against a project's capability footprint using the 5-component formula from §0.5, with only Tier 1 (0.20) and Tier 2 (0.55) confidence weights per §0.4. Produces a shortlist of 3-5 candidates with seam gap maps.

Tables touched (4): `projects`, `expert_profiles`, `expert_seam_claims`, `expert_domain_depths`

Endpoints: `POST /matching/{projectId}`, `GET /matching/{projectId}/shortlist`

ASCII Swimlane



```

[3] SELECT FROM projects
WHERE id=projectId
→ required_seams_json
→ required_domains_json
→ archetype, tier
+-----> [4] Return project footprint
+<-----

[5] SELECT experts (self-exclusion):
SELECT ep.user_id,
       ep.stack_tags_json,
       ep.engagement_model,
       ep.archetype_history_json
FROM expert_profiles ep
WHERE ep.user_id NOT IN
      (SELECT user_id FROM
       project_members WHERE
       project_id=?)
+-----> [6] Return expert pool
+<-----

[7] For each expert, fetch claims:
SELECT esc.seam_code,
       esc.verification_tier
FROM expert_seam_claims esc
WHERE esc.expert_id = ?
+-----> [8] Return seam claims
+<-----

[9] For each expert, fetch depths:
SELECT edd.domain_code,
       edd.depth_level,
       edd.verification_tier
FROM expert_domain_depths edd
WHERE edd.expert_id = ?
+-----> [10] Return domain depths
+<-----

[11] COMPOSITE SCORE CALCULATION
(NestJS or FastAPI):

Per §0.5 weights:
a. Seam alignment (40%):
   For each required_seam:
     IF expert has claim:
       Tier 1 (CLAIMED) → 0.20
       Tier 2 (EVIDENCE_BACKED) → 0.55
     IF load-bearing seam missing:
       → negative contribution
b. Domain depth coverage (25%):
   expert depth ≥ required depth?
c. Archetype-tier congruence (20%): archetype_history_json match
d. Engagement model fit (10%): model match
e. Stack tags & recency (5%): stack_tags_json overlap

HARD GATE:
claimed-to-verified ratio > 4:1
→ excluded from shortlist

Match strength labels per §0.5:
Strong > 0.78, Qualified 0.58-0.78
Conditional 0.42-0.58

[12] Build seam gap map per expert:
Per required seam:
Amber = EVIDENCE_BACKED (Tier 2)
Yellow = CLAIMED (Tier 1)
Red = Absent
(Green/Verified absent in MVP – no Tier 3/4)

[13] Rank → shortlist 3-5 candidates
Return {shortlist: [{expert_id,

```

		strength_label, gap_map, score_components}}}	
	+<-----+		
[14] CEO views shortlist with seam gap maps and strength labels (not numeric scores)			
+-----+-----+-----+			

Step Detail Table

Step	Actor	Action	Tables (CRUD)	Key Logic	Endpoint
1-2	CEO→NestJS	Initiate matching; guards	users (R), projects (R)	project must be PUBLISHED, user must be pro	POST /matching/{projectId}
3-4	NestJS	Read project footprint	projects (R)	Extract required_seams_json, required_domains_json, archetype, tier	—
5-6	NestJS	Query expert pool with self-exclusion	expert_profiles (R)	WHERE user_id NOT IN project members	—
7-10	NestJS	Fetch per-expert seam claims + domain depths	expert_seam_claims (R), expert_domain_depths (R)	verification_tier determines confidence factor (0.20 or 0.55)	—
11	NestJS/FastAPI	Compute composite scores	—	5-component formula per \$0.5; hard gate ratio > 4:1	—
12-13	NestJS	Build gap maps + rank	—	Tier 2=Amber, Tier 1=Yellow, Absent=Red	—
14	CEO	View shortlist	—	—	GET /matching/{projectId}/shortlist

MF-6: Simplified Bid & Connection Flow

Overview

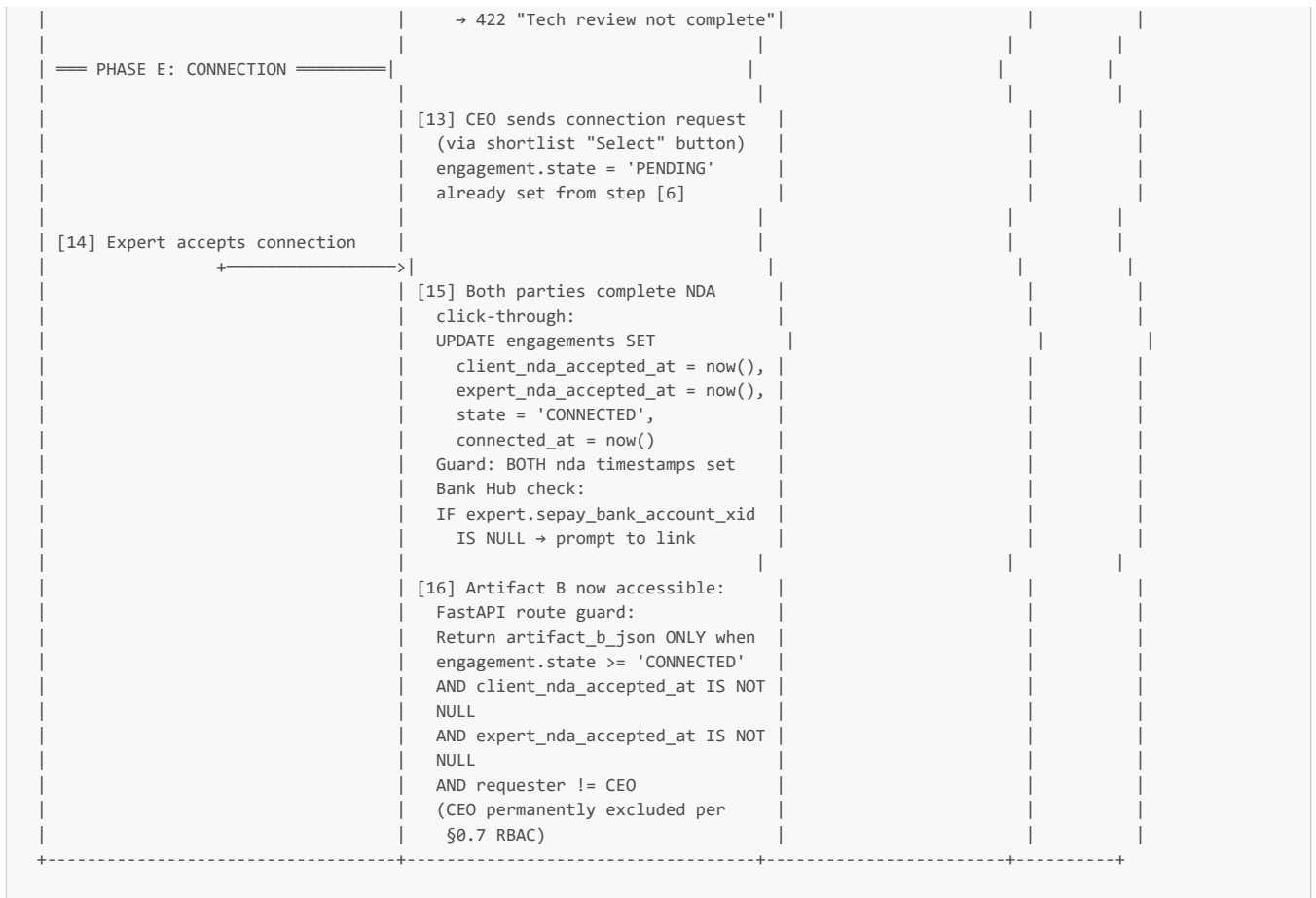
Expert views Artifact A, asks pre-bid questions via messages channel, submits a 3-component bid. TECH_TEAM reviews and sets tech_status. CEO reviews and sets ceo_status. Single mutable row — no versioned snapshots. Pre-bid questions use messages (no spec_clarifications table). Price negotiation via negotiated_price_vnd column (one round). References: \$0.6 (Bid states), \$0.7 (RBAC — TECH_TEAM approves before CEO).

Tables touched (4): capability_bids, engagements, projects, messages

Endpoints: GET /projects/{id}/artifact-a, POST /bids, PUT /bids/{id}, PUT /bids/{id}/tech-review, PUT /bids/{id}/ceo-decision, POST /engagements/{id}/connect, PUT /engagements/{id}/accept-nda

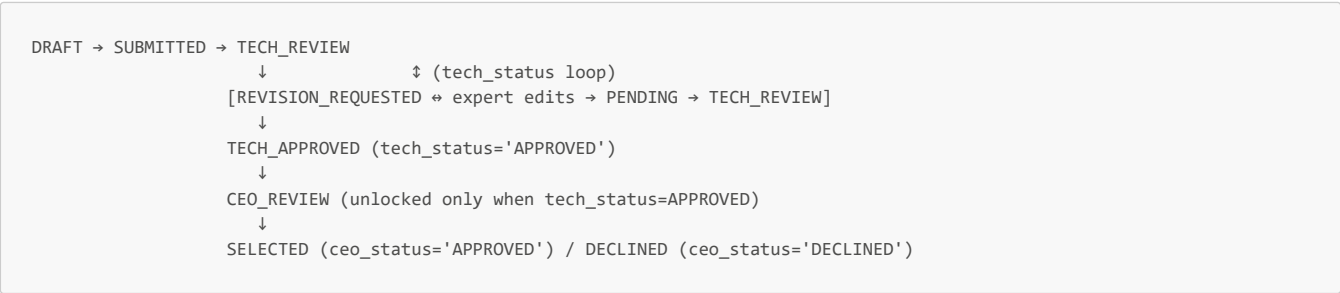
ASCII Swimlane

EXPERT	SYSTEM (NestJS)	CLIENT / TECH_TEAM	CEO
===== PHASE A: PRE-BID =====			
[1] Views PUBLISHED project → GET Artifact A (artifact_a_json on projects)			
[2] Has technical question? Posts via messages channel (no spec_clarifications table)			
+----->	[3] INSERT messages: engagement_id=NULL* (pre-bid thread uses project_id context) sender_id=expert_id content="What is current error rate baseline?"		
	+----->	[4] TECH_TEAM sees message, answers	



Step Detail Table

Step	Actor	Action	Tables (CRUD)	State Change	Endpoint
1	Expert	View Artifact A	projects (R — artifact_a_json)	—	GET /projects/{id}/artifact-a
2-3	Expert	Ask pre-bid question	messages (C)	—	POST /engagements/{id}/messages
4	TECH_TEAM	Answer via messages	messages (C)	—	POST /engagements/{id}/messages
5-6	Expert	Submit 3-component bid	engagements (C), capability_bids (C)	bid.state=SUBMITTED, tech_status=PENDING, ceo_status=PENDING	POST /bids
7-8a	TECH_TEAM	Set tech_status=REVISION_REQUESTED + tech_feedback	capability_bids (U)	tech_status: PENDING→REVISION_REQUESTED	PUT /bids/{id}/tech-review
9a	Expert	Edit bid row in place	capability_bids (U)	tech_status→PENDING, version_number++	PUT /bids/{id}
10	TECH_TEAM	Set tech_status=APPROVED	capability_bids (U)	tech_status: PENDING→APPROVED	PUT /bids/{id}/tech-review
11-12	CEO	Review + set ceo_status + optional negotiated_price_vnd	capability_bids (U)	ceo_status: PENDING→APPROVED/DECLINED	PUT /bids/{id}/ceo-decision
13	CEO	Connection request sent	engagements (R — already PENDING)	—	—
14-15	Expert	Accept + NDA click-through	engagements (U)	state: PENDING→CONNECTED; nda timestamps set	POST /engagements/{id}/connect, PUT /engagements/{id}/accept-nda
16	System	Artifact B route guard unlocks	projects (R — artifact_b_json)	—	GET /projects/{id}/artifact-b



Key scope-reduction: Mutable single row replaces `bid_versions`, `bid_revision_requests`, `price_negotiations`, `bid_conflict_overrides` tables. `tech_feedback` column replaces Surface B table. `negotiated_price_vnd` column replaces Surface C table. CEO override is implicit — CEO can APPROVE even if TECH_TEAM had REVISION_REQUESTED (no separate conflict override table needed since tech_status/ceo_status are independent columns).

MF-7: Milestone Management + DoD + Escrow (2-Layer)

Overview

CEO funds milestones via per-milestone VA, triggering escrow lock via SePay IPN. Expert creates DoD checklist, completes items, submits deliverable. Sign-off authority verifies acceptance criteria criterion by criterion. APPROVED triggers atomic ledger release + chi hộ auto-transfer. References: \$0.6 (Milestone, DoD states), \$0.8 (Payment Architecture).

Tables touched (7): `milestones`, `acceptance_criteria`, `milestone_dod_items`, `milestone_submissions`, `paygated_documents`, `escrow_accounts`, `wallet_transactions`

Endpoints: `POST /milestones`, `POST /milestones/{id}/criteria`, `POST /milestones/{id}/dod-items`, `PUT /milestones/{id}/fund`, `PUT /milestones/{id}/dod/{itemId}`, `POST /milestones/{id}/submit`, `PUT /criteria/{id}/verify`, `POST /webhooks/sepay/ipn`

ASCII Swimlane



== PHASE B: FUNDING ==

[5] CEO clicks "Fund Milestone 1"

+----->

```
[6] PUT /milestones/{id}/fund
Create per-milestone VA via SePay:
INSERT virtual_accounts:
  entity_type:'MILESTONE',
  entity_id:milestone_id,
  fixed_amount:payment_amount_vnd,
  expires_at:now()+24h,
  status:'ACTIVE'
UPDATE milestones SET
  state:'AWAITING_PAYMENT',
  va_number, va_expires_at
Return VietQR
```

+<-----

[7] Scans VietQR, transfers
exact fixed_amount via bank

SePay IPN fires ----->

```
[8] IPN MILESTONE branch:
a. Verify HMAC
b. SELECT virtual_accounts
   WHERE va_number=?
   → entity_type='MILESTONE'
   → entity_id=milestone_id
c. Validate: amount == va.fixed_
   amount (bank enforces this)
d. Idempotency: check wallet_txns
e. DB TX (atomic):
   [LEDGER: ESCROW_LOCK]
   UPDATE wallets SET
     available_balance -= amount,
     locked_balance += amount
   INSERT wallet_transactions:
     type:'ESCROW_LOCK',
     ref:'ESC_LOCK:{milestone_id}'
   INSERT escrow_accounts:
     milestone_id, amount,
     client_wallet_id,
     expert_wallet_id,
     status:'HELD'
   UPDATE milestones SET
     state:'FUNDED',
     funded_at:now()
   Auto-advance:
     state:'IN_PROGRESS'
   UPDATE paygated_documents SET
     release_state:'RELEASED'
     WHERE milestone_id=?
f. Return 200 OK to SePay

[9] IF first milestone funded:
UPDATE engagements SET
  state:'ACTIVE'
(CONNECTED → ACTIVE)
```

== PHASE C: DoD CHECKLIST ==

[10] Expert
creates DoD
checklist:

POST
/milestones
/{id}/dod-
items

```
[11] INSERT milestone_dod_items:
  milestone_id, item_description,
  is_required:true,
  status:'PENDING'
```

[12] Expert
marks items
COMPLETED

```
[13] PUT /milestones/{id}/dod/
    {itemId}
UPDATE milestone_dod_items SET
```

	status:'COMPLETED', completed_at:now(), completion_note:{text} (IF is_required=true: note req'd) DB CHECK enforced: NOT(is_required=true AND status='NOT_APPLICABLE')			
== PHASE D: SUBMIT DELIVERABLE ==				
			[14] Expert submits deliverable	
	[15] POST /milestones/{id}/submit GUARD: SELECT COUNT(*) FROM milestone_dod_items WHERE milestone_id=? AND is_required=true AND status!='COMPLETED' IF >0 → 422 REQUIRED_DOD_INCOMPLETE with list of unchecked items INSERT milestone_submissions: milestone_id, expert_id, description, files_json, submitted_at:now() UPDATE milestones SET state:'SUBMITTED', submitted_at:now()			
== PHASE E: SIGN-OFF ==				
			[16] TECH_TEAM verifies criteria criterion by criterion:	
	[17] PUT /criteria/{id}/verify UPDATE acceptance_criteria SET verified_at:now() OR write revision_note (reject) → milestone state: IN_REVISION			
	[18] APPROVED GUARD: SELECT COUNT(*) FROM acceptance_criteria WHERE milestone_id=? AND is_required=true AND verified_at IS NULL IF >0 → 422 UNVERIFIED_CRITERIA with list of unverified criteria			
	[19] ALL required criteria verified → APPROVED – atomic ledger release: DB TX: [LEDGER: ESCROW_RELEASE] UPDATE wallets (client) SET locked_balance -= amount [LEDGER: PLATFORM_FEE] fee = amount * platform_fee_pct (READ from platform_settings, NOT hardcoded) UPDATE wallets (platform) SET available_balance += fee [LEDGER: CREDIT_EXPERT] net = amount - fee UPDATE wallets (expert) SET available_balance += net INSERT wallet_transactions (3 rows): ESCROW_RELEASE, PLATFORM_FEE, (expert credit via ESCROW_RELEASE) UPDATE escrow_accounts SET status:'RELEASED', released_at:now() UPDATE milestones SET state:'APPROVED', approved_at:now() COMMIT → chi hệ API called (async) [API] POST SePay chi hệ: {amount:net, bank_account_xid, reference:"WD-{withdrawal_id}"}			

		INSERT withdrawal_requests: type: 'MILESTONE_RELEASE', status: 'PENDING'			
		[20] SePay credit IPN → withdrawal COMPLETED			
		UPDATE milestones SET state: 'RELEASED', released_at: now()			
		[21] IF all milestones RELEASED: UPDATE engagements SET state: 'CLOSED'			
+-----+-----+-----+-----+					

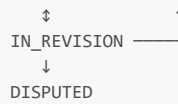
Step Detail Table

Step	Actor	Action	Tables (CRUD)	State Change	Endpoint
1-2	CEO	Create milestone	milestones (C)	state: DEFINED	POST /milestones
3-4	CEO→FastAPI	Define criteria + LLM quality gate	acceptance_criteria (C), platform_decisions (C)	verified_at: NULL	POST /milestones/{id}/criteria
5-6	CEO	Fund milestone → VA created	virtual_accounts (C), milestones (U)	state: DEFINED→AWAITING_PAYMENT	PUT /milestones/{id}/fund
7-8	CEO→SePay→NestJS	Scan QR → IPN → escrow lock	virtual_accounts (R), wallets (U), wallet_transactions (C), escrow_accounts (C), milestones (U), paygated_documents (U)	state: AWAITING_PAYMENT→FUNDED→IN_PROGRESS; escrow: HELD	POST /webhooks/sepay/ipn
9	NestJS	First milestone → engagement ACTIVE	engagements (U)	state: CONNECTED→ACTIVE	—
10-11	Expert	Create DoD items	milestone_dod_items (C)	status: PENDING	POST /milestones/{id}/dod-items
12-13	Expert	Complete DoD items	milestone_dod_items (U)	status: PENDING→COMPLETED	PUT /milestones/{id}/dod/{itemId}
14-15	Expert	Submit deliverable (DoD guard)	milestone_submissions (C), milestones (U)	state: IN_PROGRESS→SUBMITTED	POST /milestones/{id}/submit
16-17	TECH_TEAM	Verify criteria	acceptance_criteria (U)	verified_at set (or revision_note written)	PUT /criteria/{id}/verify
18-19	System	All required criteria verified → APPROVED + ledger release	wallets (U ×3), wallet_transactions (C ×3), escrow_accounts (U), milestones (U), withdrawal_requests (C)	state: SUBMITTED→APPROVED; escrow: HELD→RELEASED	—
20	SePay→NestJS	Chi hộ credit IPN	withdrawal_requests (U), milestones (U)	state: APPROVED→RELEASED; withdrawal: COMPLETED	POST /webhooks/sepay/ipn
21	NestJS	All milestones RELEASED → engagement CLOSED	engagements (U)	state: ACTIVE→CLOSED	—

Milestone State Machine (§0.6 — Scope-Reduced)

--

DEFINED → AWAITING_PAYMENT → FUNDED → IN_PROGRESS → SUBMITTED → APPROVED → RELEASED



MF-8: Dispute Resolution (2-Layer)

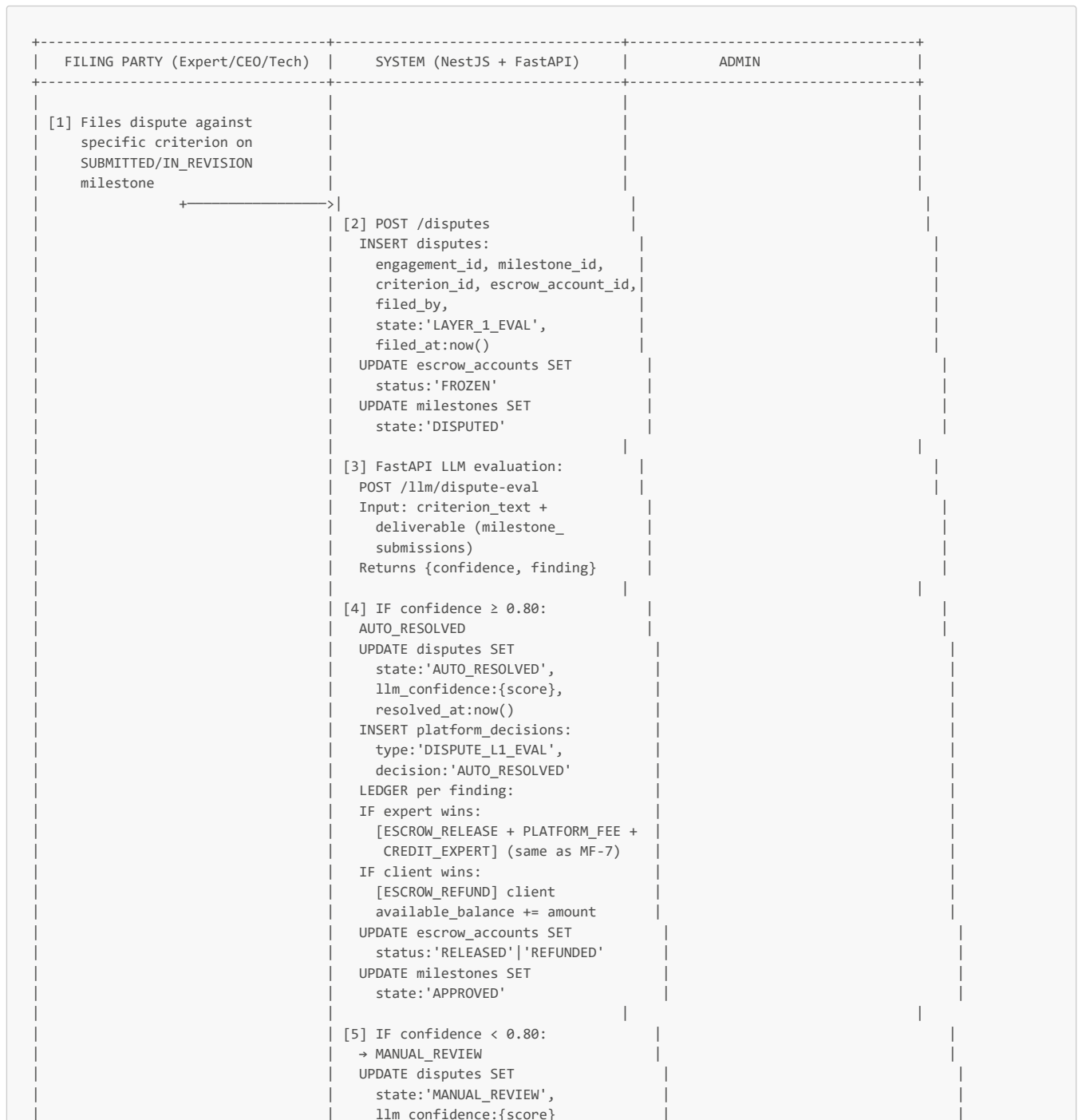
Overview

Dispute filed against a specific acceptance criterion. Layer 1: LLM evaluates criterion vs. deliverable. If confidence $\geq 0.80 \rightarrow$ AUTO_RESOLVED. If $< 0.80 \rightarrow$ MANUAL_REVIEW $\rightarrow$ admin resolves via dashboard button. References: \$0.6 (Dispute states), \$0.8 (Ledger operations).

Tables touched (5): `disputes`, `escrow_accounts`, `wallet_transactions`, `wallets`, `platform_decisions`

Endpoints: `POST /disputes`, `PUT /admin/disputes/{id}/resolve`

ASCII Swimlane





Dispute State Machine (2-Layer — \$0.6)



Key scope-reduction: No Layer 2 mutual agreement (48h cooling window) or Layer 3 automatic 50/50 split. Admin resolves Layer 2 manually with 3 options. No `dispute_resolution_reports` table.

Group 3 — Path B: Service Marketplace

MF-9: Expert Service Publishing (AI Generator)

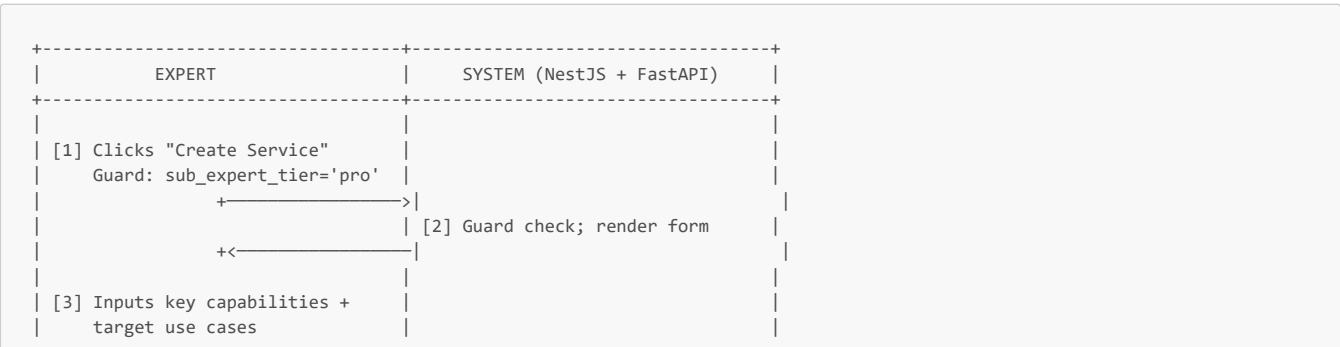
Overview

Expert (Pro) uses AI Service Generator to create a service listing. Publishes to marketplace. References: \$0.9 (Expert Pro required for AI Service Generator).

Tables touched (2): `services`, `expert_profiles`

Endpoints: `POST /services/generate`, `POST /services`, `PUT /services/{id}`

ASCII Swimlane





MF-10: Client Buys Service / Tech Discovery

Overview

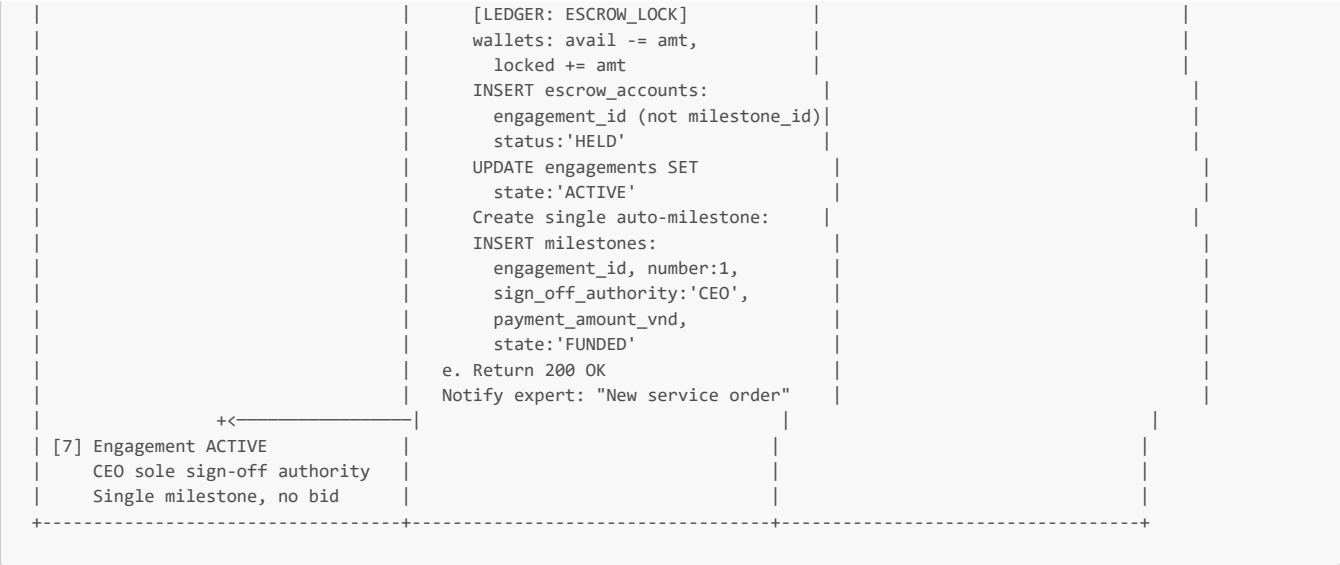
CEO browses marketplace, purchases a service via per-order VA payment. Creates SERVICE_PURCHASE or TECH_DISCOVERY engagement directly in ACTIVE state (no bid, no elicitation). References: \$0.6 (Engagement types), \$0.8 (Payment Architecture).

Tables touched (5): `services`, `engagements`, `virtual_accounts`, `escrow_accounts`, `wallet_transactions`

Endpoints: `GET /services`, `POST /services/{id}/purchase`, `POST /webhooks/sepay/ipn`

ASCII Swimlane





Key structural difference from Path A: No elicitation, no matching, no bid, no TECH_TEAM involvement. Single fixed-price milestone with CEO-only sign-off. `engagements.project_id = NULL, engagements.service_id NOT NULL`. Enforced by table-level CHECK constraint.

Group 4 — Financial Flows

MF-11: Wallet Top-Up

Overview

User scans permanent WALLET_TOPUP VA VietQR. SePay IPN credits wallet. Pure internal ledger — no external API calls.

Tables touched (3): `virtual_accounts`, `wallets`, `wallet_transactions`

Endpoints: `POST /wallet/topup-qr`, `POST /webhooks/sepay/ipn`

ASCII Swimlane




```
| [6] Balance updated | | |
+-----+-----+-----+
```

Idempotency guarantee: `wallet_transactions` has `UNIQUE INDEX wallet_tx_idempotency ON (wallet_id, reference_id) WHERE reference_id IS NOT NULL`. SePay IPN retries cannot double-credit.

MF-12: Expert Withdrawal (Chi Hộ)

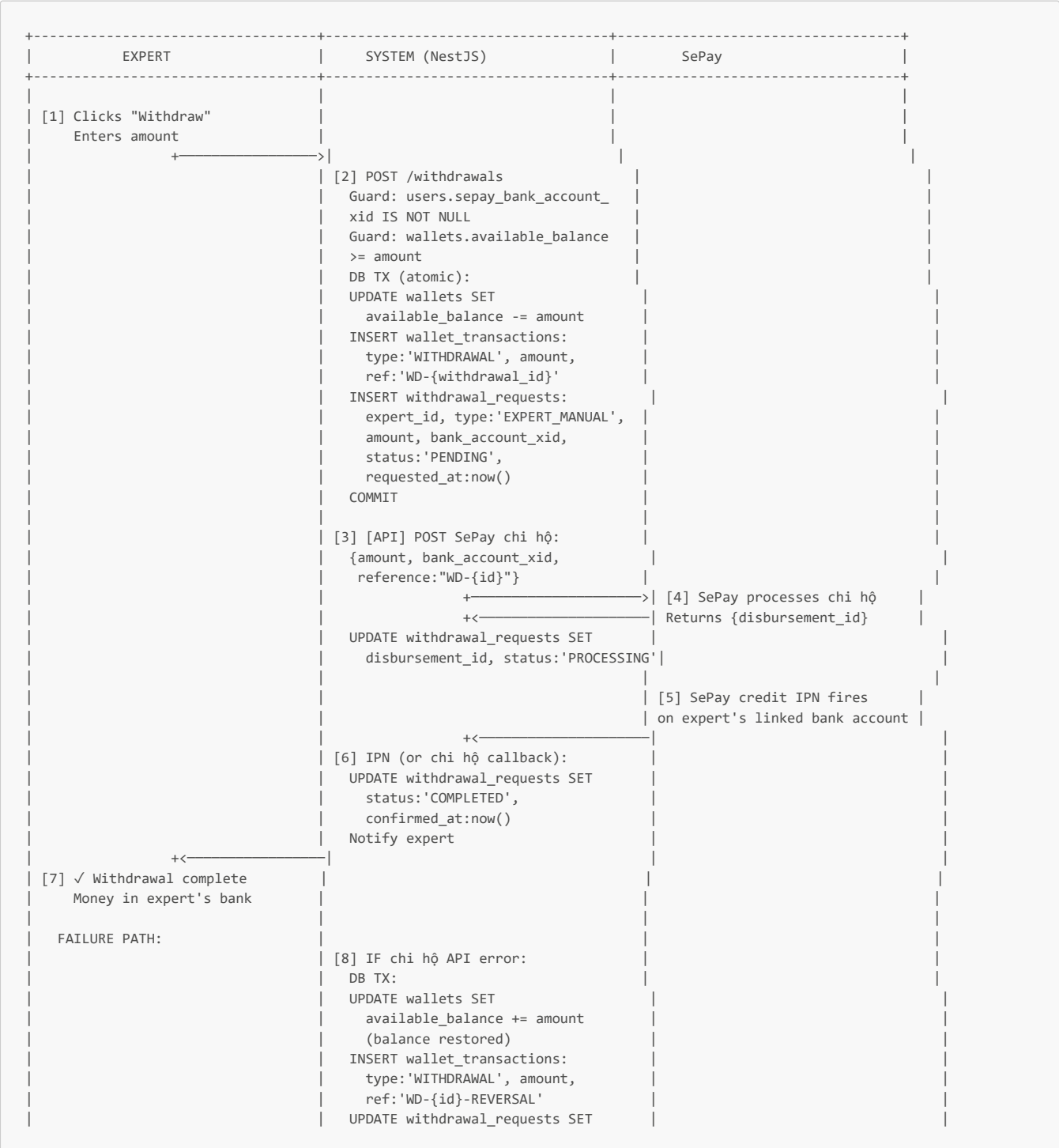
Overview

Expert requests cash-out. System debits wallet atomically, calls SePay chi hộ API. Credit IPN confirms completion. Zero admin involvement.

Tables touched (3): `wallets`, `wallet_transactions`, `withdrawal_requests`

Endpoints: `POST /withdrawals`, `POST /webhooks/sepay/ipn`

ASCII Swimlane



	status: 'FAILED'	
	Notify expert with error details	
+-----+-----+-----+		

MF-13: Subscription Purchase from Wallet

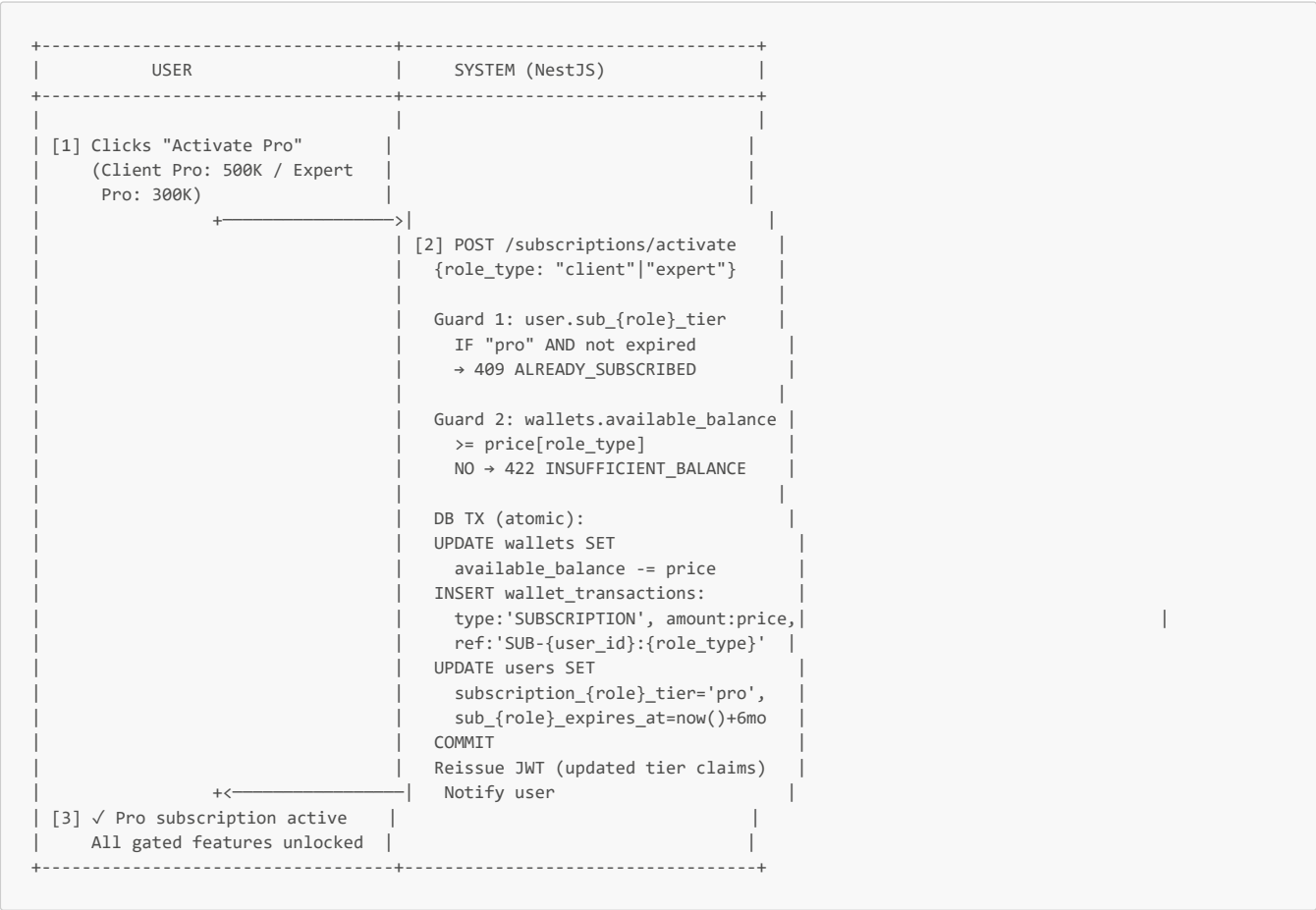
Overview

Pure internal wallet deduction. No VA, no IPN. Atomic transaction debits wallet and updates user subscription columns.

Tables touched (3): `users`, `wallets`, `wallet_transactions`

Endpoints: `POST /subscriptions/activate`

ASCII Swimlane



Key scope-reduction: No `user_subscriptions` table. Subscription state stored as `users.subscription_client_tier`, `users.sub_client_expires_at`, `users.subscription_expert_tier`, `users.sub_expert_expires_at`. The `wallet_transactions` reference_id pattern `SUB-{user_id}:{role_type}` provides the audit trail that the cut table would have provided.

Group 5 — Platform-Specific Flows

MF-14: Pay-Gated Document Release (IPN-Triggered)

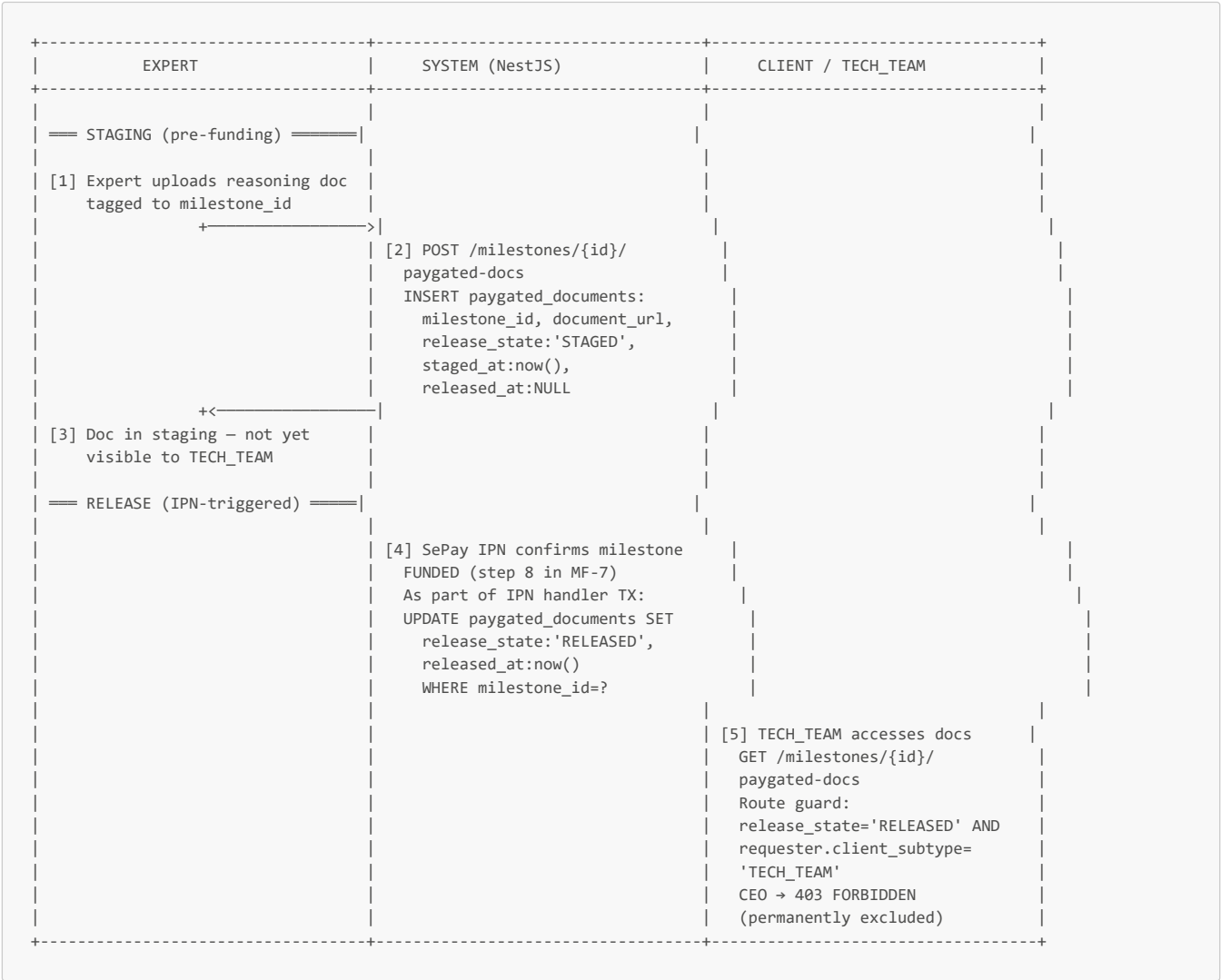
Overview

Core F5 IP deadlock resolution mechanism. Expert stages reasoning documents tagged to a milestone. When SePay IPN confirms milestone FUNDED, documents auto-release to `TECH_TEAM`. CEO permanently excluded. References: \$0.7 (RBAC — `TECH_TEAM`-only access).

Tables touched (2): `paygated_documents`, `milestones`

Endpoints: `POST /milestones/{id}/paygated-docs`, `GET /milestones/{id}/paygated-docs`

ASCII Swimlane



MF-15: Dual-Role Account Switch

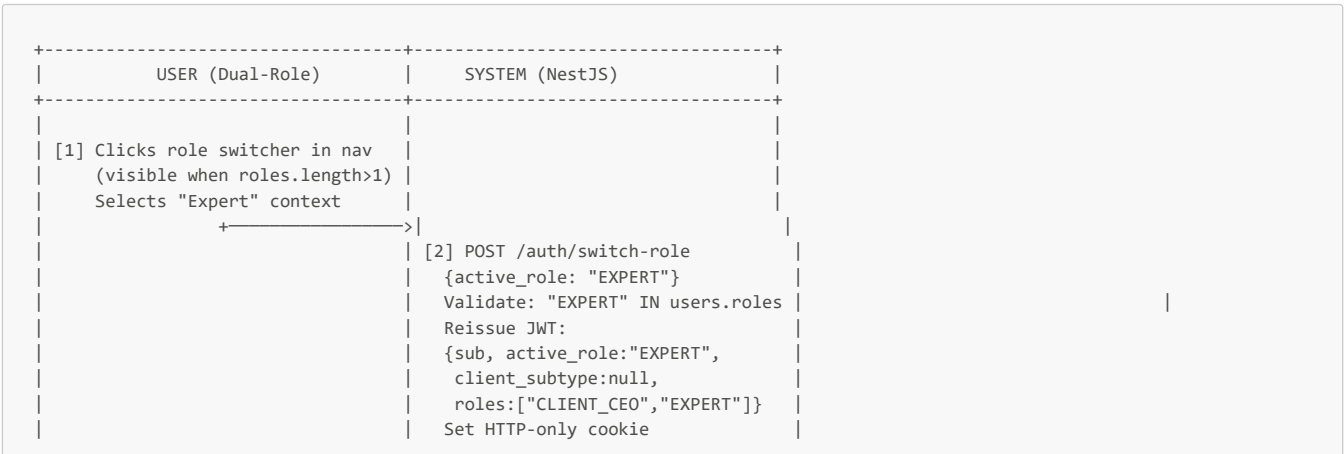
Overview

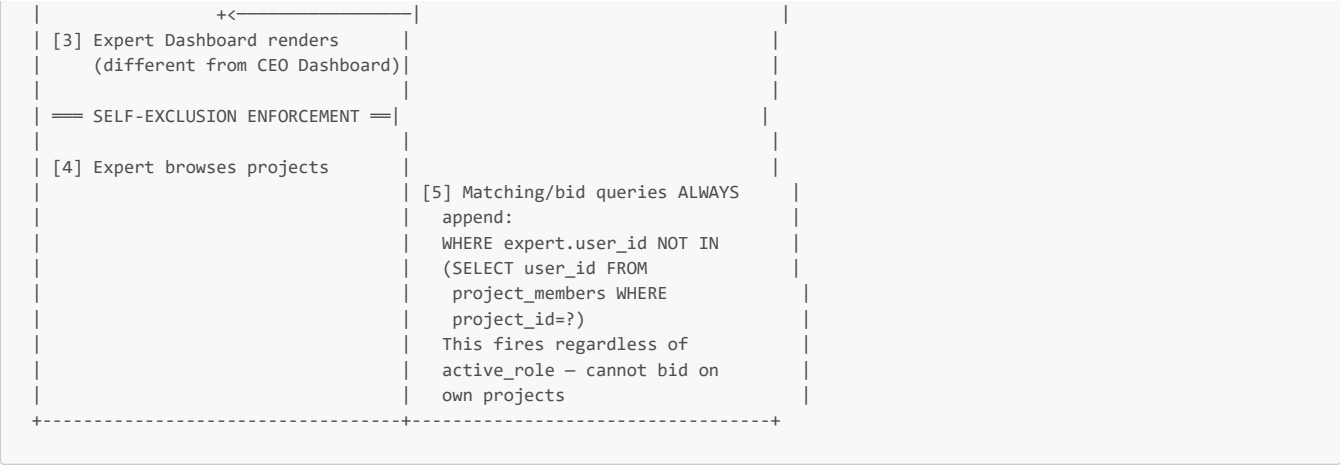
Single account holds `roles: ["CLIENT_CEO", "EXPERT"]`. Role switcher reissues JWT with new `active_role`. Self-exclusion always applies. References: \$0.7 (Dual-role, self-exclusion).

Tables touched (1): `users`

Endpoints: `POST /auth/switch-role`

ASCII Swimlane





MF-16: Expert Portfolio Tier 2 Auto-Upgrade

Overview

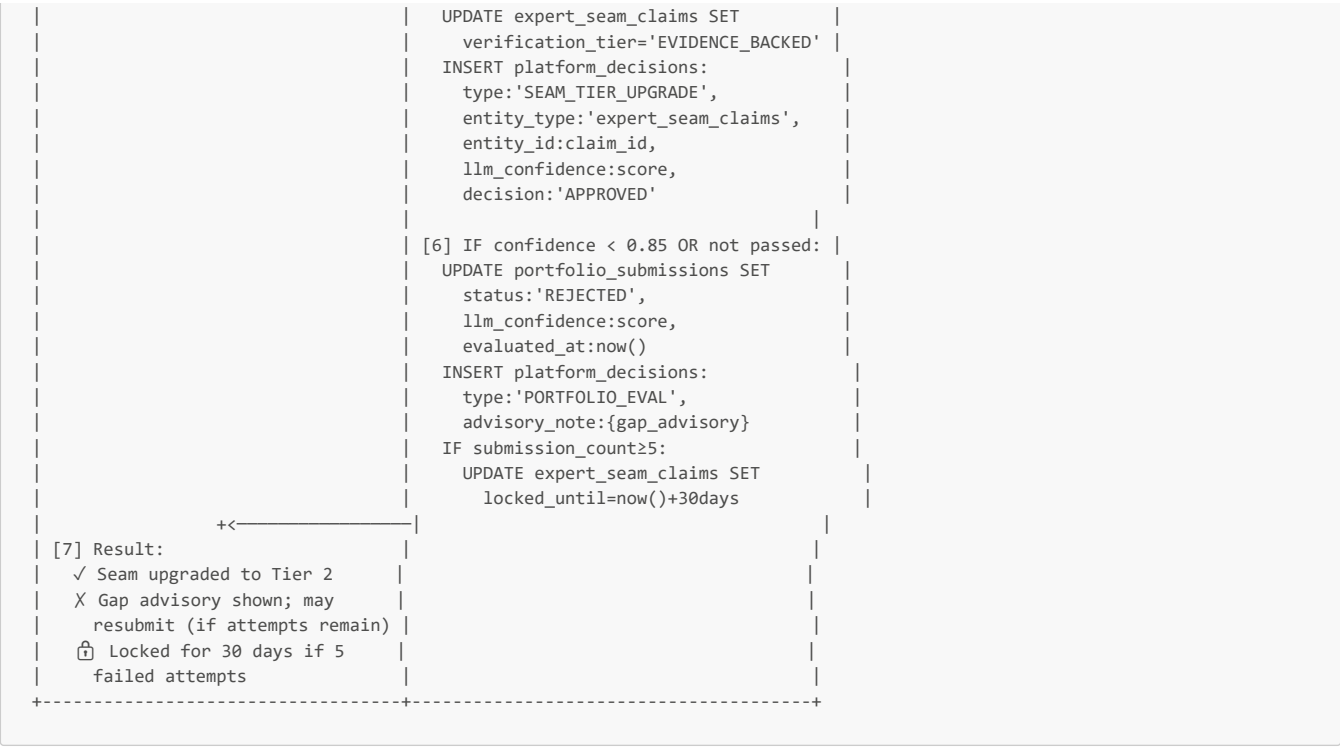
Dedicated flow for the LLM portfolio verification pipeline. Expert submits structured decision-point writeup. FastAPI evaluates. Auto-upgrades seam claim to EVIDENCE_BACKED if confidence ≥ 0.85 . 5-attempt / 30-day lockout throttle. References: \$0.4 (2-Tier Verification).

Tables touched (3): `portfolio_submissions`, `expert_seam_claims`, `platform_decisions`

Endpoints: `POST /portfolio-submissions`, `GET /portfolio-submissions/{id}/eval-result`

ASCII Swimlane





Group 6 — Admin Flows

MF-17: Platform Integrity Monitor & Analytics

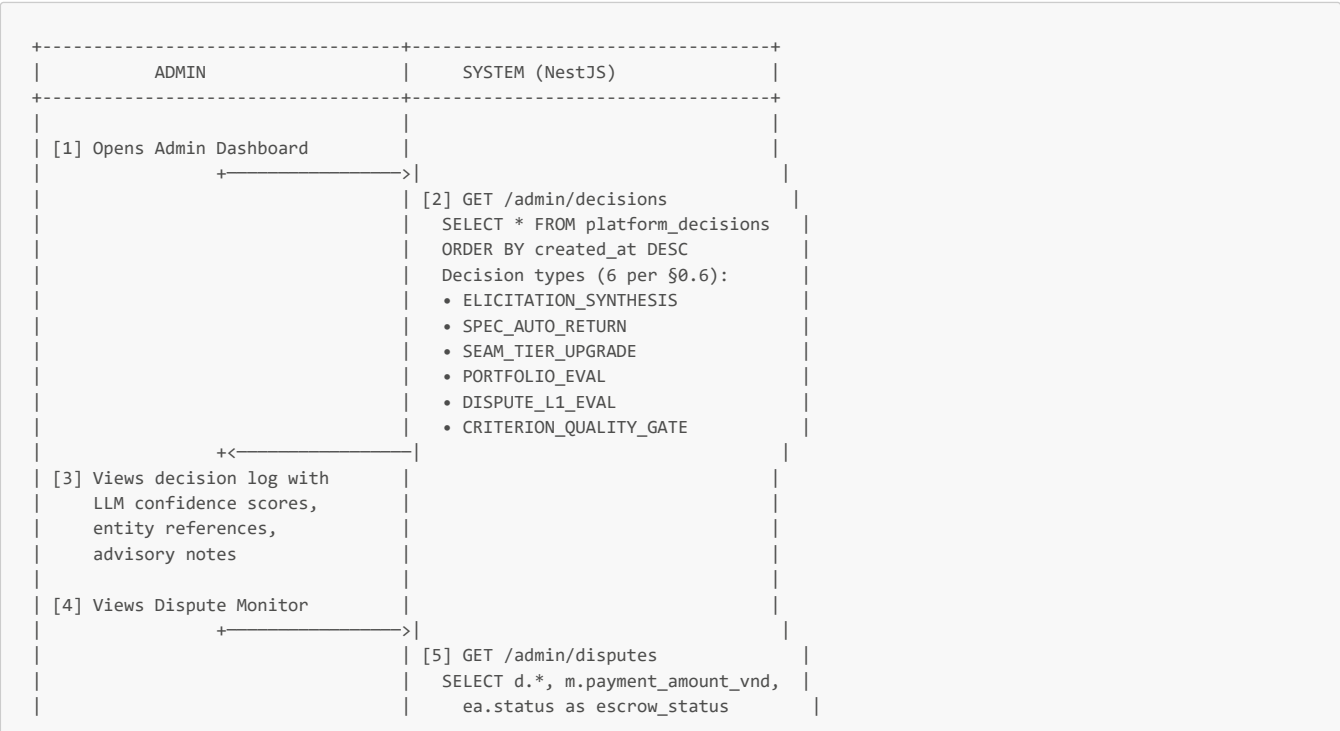
Overview

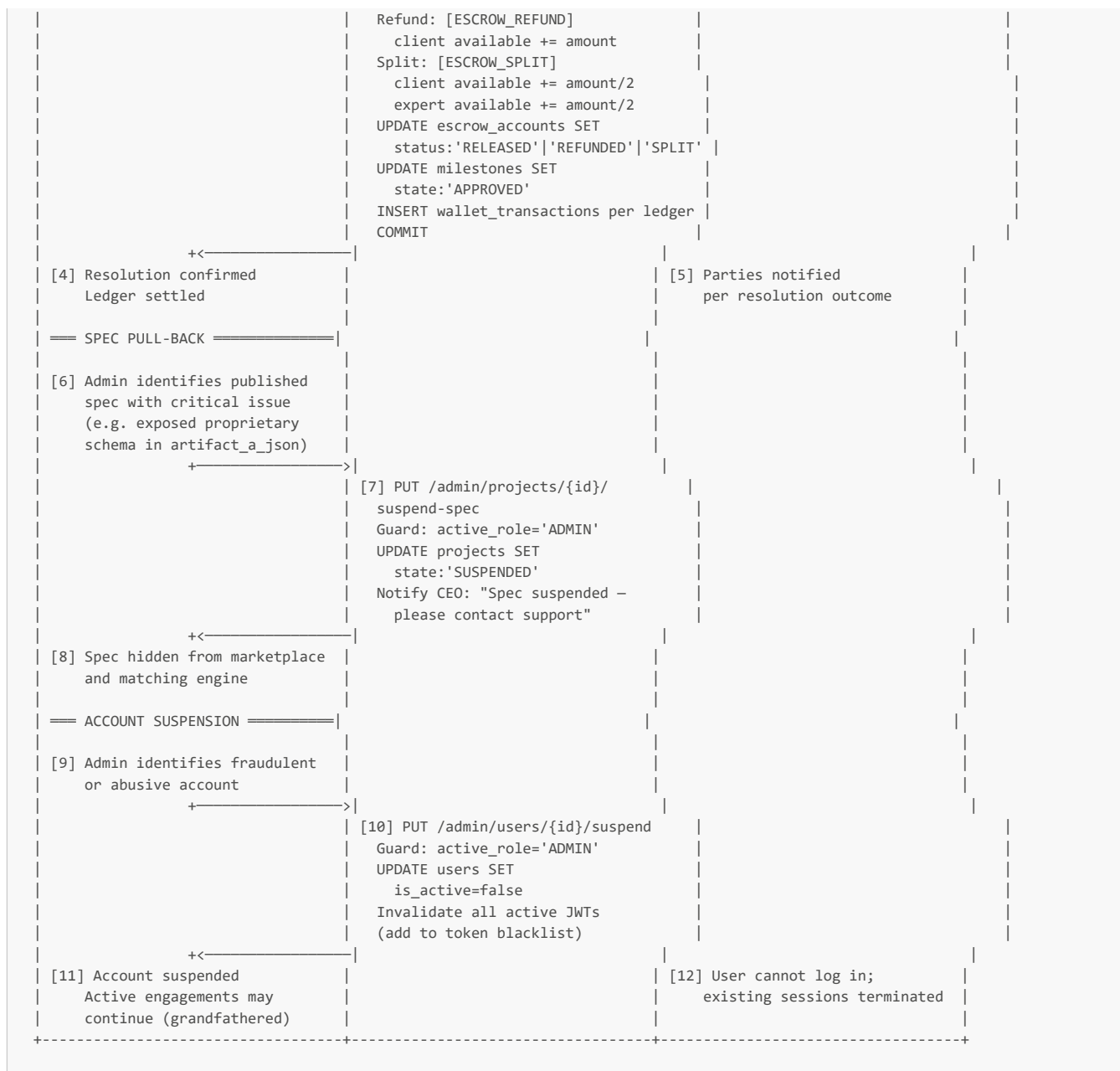
Read-only monitoring dashboard. Displays `platform_decisions` log, auto-upgrade/return history, dispute log, transaction ledger, analytics. References: §0.7 (ADMIN — read-only monitoring).

Tables touched (1 — read only): `platform_decisions` (plus read-only access to `wallet_transactions`, `disputes`, `escrow_accounts`, `users`, `withdrawal_requests`)

Endpoints: `GET /admin/decisions`, `GET /admin/disputes`, `GET /admin/transactions`, `GET /admin/analytics`

ASCII Swimlane





Cross-Table Ledger Operations Summary

Every financial action in the platform resolves to one of these atomic wallet transaction patterns:

Transaction Type	Wallet Debit	Wallet Credit	Trigger	Flow
TOP_UP	—	available_balance += amount	SePay IPN (WALLET_TOPUP VA)	MF-11
SUBSCRIPTION	available_balance -= price	—	User activates Pro	MF-13
ESCROW_LOCK	available_balance -= amount	locked_balance += amount	SePay IPN (MILESTONE/SERVICE VA)	MF-7, MF-10
ESCROW_RELEASE	locked_balance -= gross	expert.available_balance += net	All required criteria verified	MF-7
PLATFORM_FEE	—	platform.available_balance += fee	Deducted on release	MF-7
ESCROW_REFUND	locked_balance -= amount	client.available_balance += amount	Dispute: client wins	MF-8, MF-18
ESCROW_SPLIT	locked_balance -= amount	Both += amount/2	Admin 50/50 split	MF-18
WITHDRAWAL	expert.available_balance -= amount	—	Expert cash-out request	MF-12

Every row in `wallet_transactions` is immutable. The `UNIQUE INDEX wallet_tx_idempotency ON (wallet_id, reference_id) WHERE reference_id IS NOT NULL` guarantees SePay IPN retries cannot double-credit.

Platform fee is never hardcoded: `SELECT platform_fee_pct FROM platform_settings LIMIT 1` is read at transaction time. Default 0.05 (5%). Can be changed via DB without a code deploy.

Appendix: 28-Table Coverage Matrix

Every table in the schema is exercised by at least one main flow. This matrix proves full CRUD coverage.

#	Table	Created By	Read By	Updated By	Primary Flows
1	users	MF-1, MF-2, MF-3	MF-1, MF-2, MF-5, MF-13, MF-15	MF-1 (sub tier), MF-2 (bank link), MF-13 (sub tier)	All
2	client_profiles	MF-1	—	—	MF-1
3	expert_profiles	MF-2	MF-5, MF-9	MF-2 (stack tags, archetype)	MF-2, MF-5
4	tech_team_profiles	MF-3	MF-3		